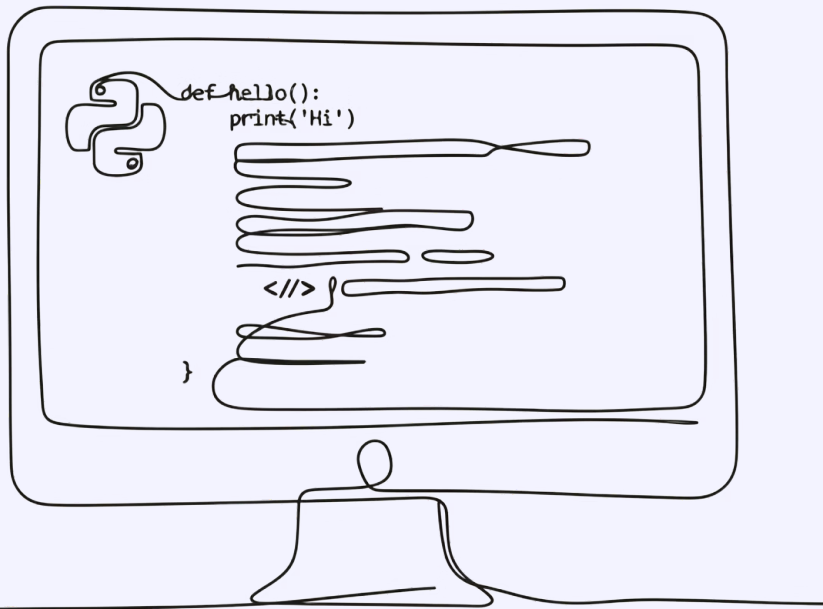


Clases y Otros Elementos en Python para el Desarrollo de Aplicaciones Web

UNIDAD 2 TEMA 3.3

MSC.ERICKA HERNÁNDEZ MENA



¿Por qué clases en el desarrollo web?

Las clases son el pilar central de la POO. Permiten **modelar entidades del mundo real** –como usuarios, productos o servicios– con atributos y comportamientos propios. En el desarrollo web, esto se traduce en código organizado, reutilizable y fácil de mantener.



Organización

Estructura el código en módulos coherentes y bien definidos.



Reutilización

Hereda y extiende funcionalidades sin reescribir código.



Escalabilidad

Facilita el crecimiento del proyecto sin perder el control.



Seguridad

Encapsula datos sensibles para protegerlos del acceso externo.

Nombres y Objetos en Python

En Python, **todo es un objeto**. Un *nombre* es simplemente una referencia que apunta a un objeto almacenado en memoria. Varios nombres pueden apuntar al mismo objeto.

Un nombre → Un objeto

```
nombre = "Ericka"  
# "nombre" apunta al objeto "Ericka"
```

Aquí `nombre` es la referencia y `"Ericka"` es el objeto en memoria.

Múltiples nombres → Mismo objeto

```
a = [1, 2, 3]  
b = a  
# "b" apunta al mismo objeto que "a"
```

Modificar `a` también afecta a `b`, ya que comparten la misma referencia en memoria.

Ámbito y Espacios de Nombres

Un **espacio de nombres** es el conjunto de identificadores válidos dentro de un contexto. El **ámbito** define la región del programa donde una variable es accesible. Comprender esto evita errores comunes en proyectos web.

Ámbito Global

```
mensaje = "Hola"

def mostrar():
    print(mensaje)

mostrar() # → "Hola"
```

La variable `mensaje` está disponible dentro y fuera de la función.

Ámbito Local

```
def mostrar():
    texto = "Bienvenido"
    print(texto)

mostrar() # → "Bienvenido"
# "texto" no existe fuera
```

La variable `texto` solo existe dentro de la función donde fue declarada.

📄 Regla LEGB: Python busca variables en orden Local → Enclosing → Global → Built-in.

Objetos de Clase e Instancia

Una **clase** actúa como una plantilla. Cada objeto creado a partir de ella es una **instancia** independiente con su propio espacio en memoria.

Definir una clase

```
class Estudiante:  
    pass
```

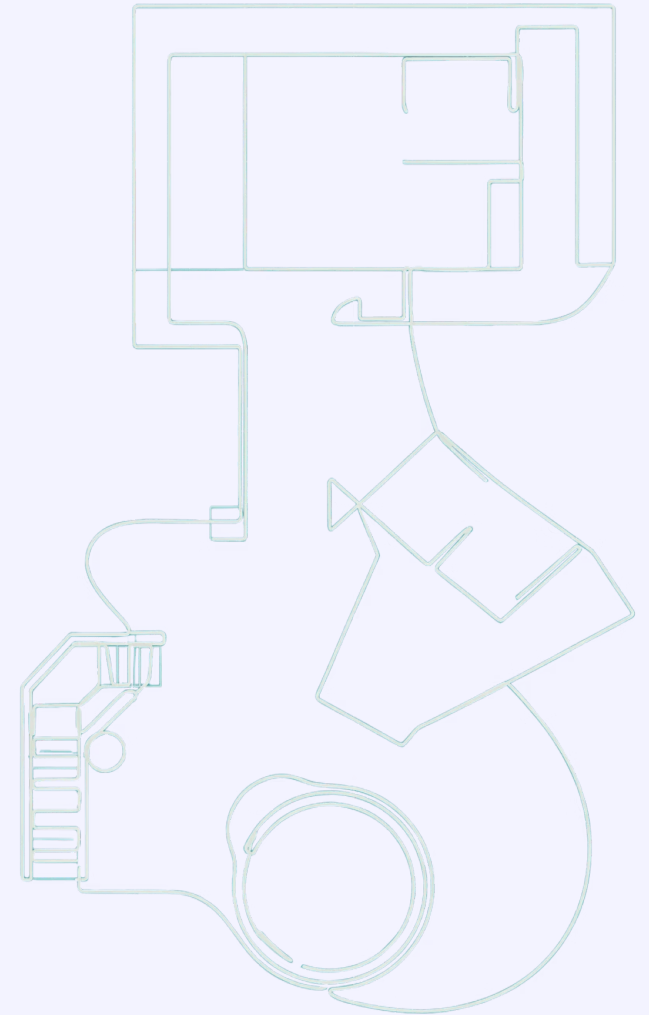
Clase vacía pero válida

Crear instancias

```
alumno1 = Estudiante()  
alumno2 = Estudiante()
```

Cada uno tiene su propia
zona de memoria

Piensa en la clase como el *molde* y en las instancias como los *objetos concretos* fabricados con ese molde. En una app web, podría ser el molde `Usuario` con instancias como `usuario1 = Usuario()`.



Métodos: Comportamiento de los Objetos

Los **métodos** son funciones definidas dentro de una clase que describen los comportamientos del objeto. El parámetro `self` hace referencia a la instancia actual.

```
class Estudiante:

    def saludar(self):
        print("Hola estudiante")

# Uso:
alumno = Estudiante()
alumno.saludar() # → "Hola estudiante"
```

`self`

Referencia obligatoria al objeto que llama al método.

`__init__`

Método especial que se ejecuta automáticamente al crear una instancia.

Retorno

Los métodos pueden devolver valores con `return`.

Variables de Clase vs. Variables de Instancia

Python distingue entre atributos **compartidos** (de clase) y atributos **proprios de cada objeto** (de instancia). Comprender la diferencia es clave para diseñar modelos correctos en aplicaciones web.

Variable de Clase (compartida)

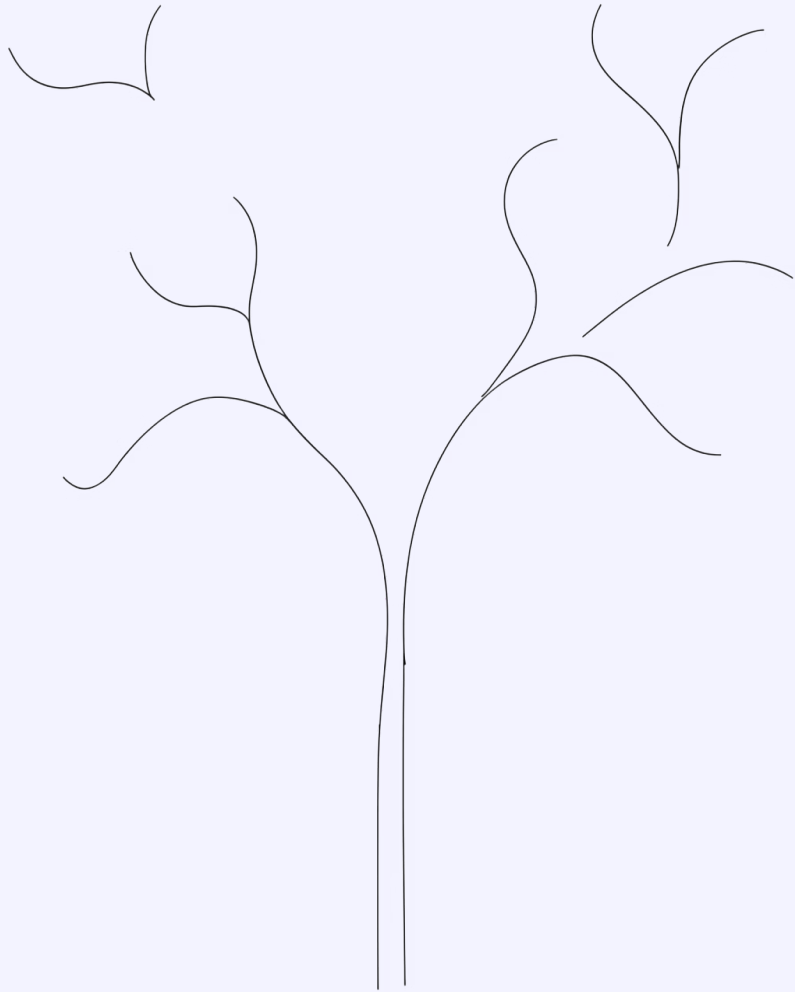
```
class Estudiante:  
    institucion = "Colegio Técnico"  
  
# Todos los objetos comparten  
# el mismo valor
```

Útil para datos comunes a todos los objetos, como el nombre de la institución o una versión de API.

Variable de Instancia (propia)

```
class Estudiante:  
    def __init__(self, nombre):  
        self.nombre = nombre  
  
a = Estudiante("Ana")  
b = Estudiante("Luis")  
# a.nombre = "Ana"  
# b.nombre = "Luis"
```

Cada objeto guarda su propio valor, independiente de los demás.



Herencia en Python

La **herencia** permite que una clase hija adquiera los atributos y métodos de una clase padre, evitando la duplicación de código y favoreciendo la reutilización en proyectos grandes.

```
class Persona:
    def hablar(self):
        print("Puede hablar")

class Estudiante(Persona):
    pass

alumno = Estudiante()
alumno.hablar() # → "Puede hablar"
```



Reutilización

No necesitas reescribir los métodos que ya existen en la clase padre.



Organización

Estructura jerarquías lógicas como Persona → Estudiante → Becario.



Mantenimiento

Un cambio en la clase padre se propaga automáticamente a todas las clases hijas.

Variables Privadas: Encapsulamiento

Las **variables privadas** protegen datos sensibles de un objeto usando el prefijo `__` (doble guion bajo). No son accesibles directamente desde fuera de la clase, lo cual es fundamental en aplicaciones web que manejan información confidencial.

Definir y acceder al saldo privado

```
class Cuenta:

    def __init__(self):
        self.__saldo = 1000

    def mostrar_saldo(self):
        return self.__saldo

c = Cuenta()
# c.__saldo →
```

Iteradores y Generadores

Los **iteradores** permiten recorrer colecciones elemento a elemento. Los **generadores** producen valores bajo demanda con `yield`, consumiendo mucha menos memoria. Ambos son esenciales para procesar grandes volúmenes de datos en aplicaciones web.

Iterador

```
numeros = [1, 2, 3]
iterador = iter(numeros)

print(next(iterador)) # → 1
print(next(iterador)) # → 2
print(next(iterador)) # → 3
```

Ideal para recorrer resultados de bases de datos fila por fila.

Generador

```
def contador():
    for i in range(5):
        yield i

for numero in contador():
    print(numero)
# → 0, 1, 2, 3, 4
```

Produce valores uno a uno sin cargar toda la secuencia en memoria.

Bases de datos

Recorre filas sin cargarlas todas a la vez.

Streams de datos

Procesa archivos grandes de forma eficiente.

APIs REST

Pagina resultados con menor uso de RAM.

Plantillas (Templates) con Jinja2

Las **plantillas** separan la lógica del programa de la presentación visual. **Jinja2**, el motor de plantillas usado por Flask y Django, permite insertar variables Python directamente en HTML para generar páginas dinámicas.

Template HTML (Jinja2)

```
<h1>Bienvenido {{ nombre }}  
</h1>
```

Código Python

```
nombre = "Ericka"
```

Resultado renderizado

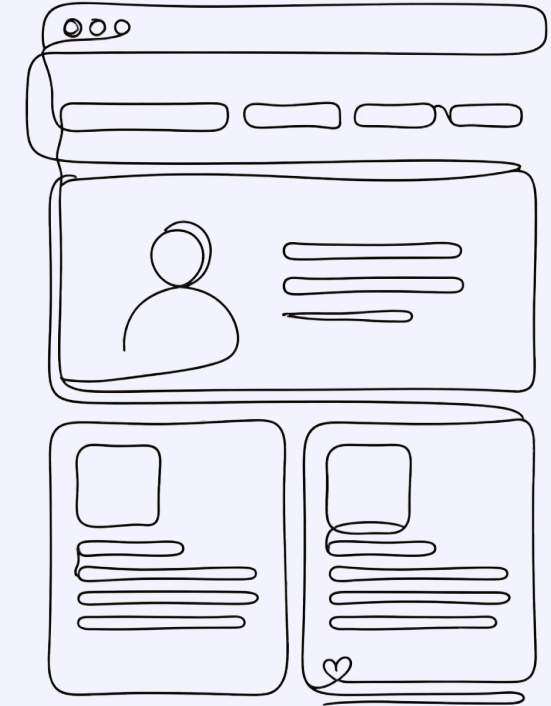
```
<h1>Bienvenido Ericka</h1>
```

Frameworks que las usan

- **Flask**: ligero, flexible, usa Jinja2 por defecto.
- **Django**: robusto, con su propio lenguaje de plantillas.

Ventajas

- Diseñadores editan HTML sin tocar Python.
- Lógica y presentación claramente separadas.
- Reutilización de layouts con herencia de templates.



Multihilos (Multithreading)

El módulo `threading` de Python permite **ejecutar varias tareas de forma simultánea**, mejorando el rendimiento de aplicaciones web que deben atender múltiples solicitudes al mismo tiempo.

```
import threading

def tarea():
    print("Ejecutando tarea")

hilo = threading.Thread(target=tarea)
hilo.start()
```



Descarga de archivos

Múltiples descargas en paralelo sin bloquear la interfaz.



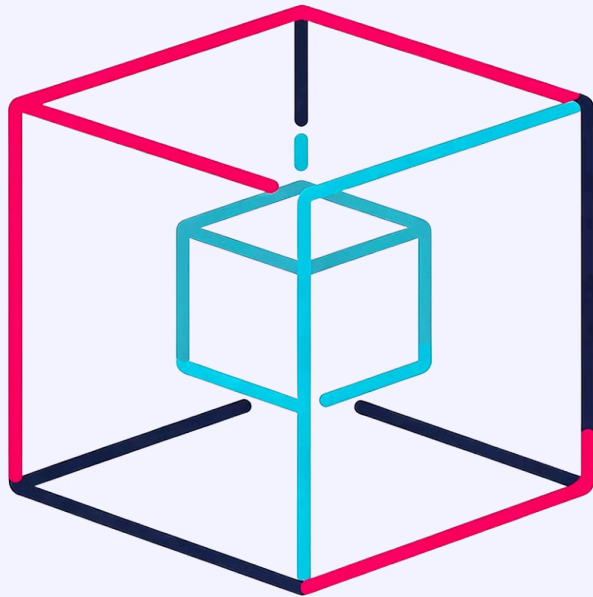
Sistemas de chat

Gestiona conexiones simultáneas de muchos usuarios.



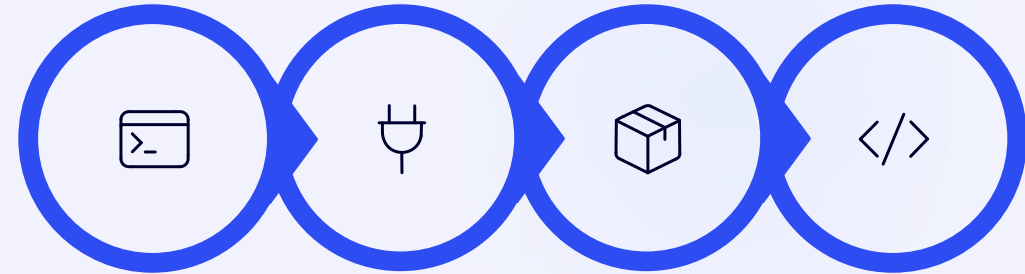
Servicios en segundo plano

Envío de correos, procesamiento de imágenes, notificaciones.



Entornos Virtuales

Un **entorno virtual** crea un espacio aislado para cada proyecto Python, evitando conflictos entre versiones de bibliotecas. Es una práctica esencial en el desarrollo web profesional.



Crear venv

Activar

Instalar

Desarrollar

Usar entornos virtuales garantiza que cada proyecto tenga sus propias dependencias, sin interferir con otros proyectos ni con el Python global del sistema.

 Windows

```
mi_entorno\Scripts\activate
```

 Linux / Mac

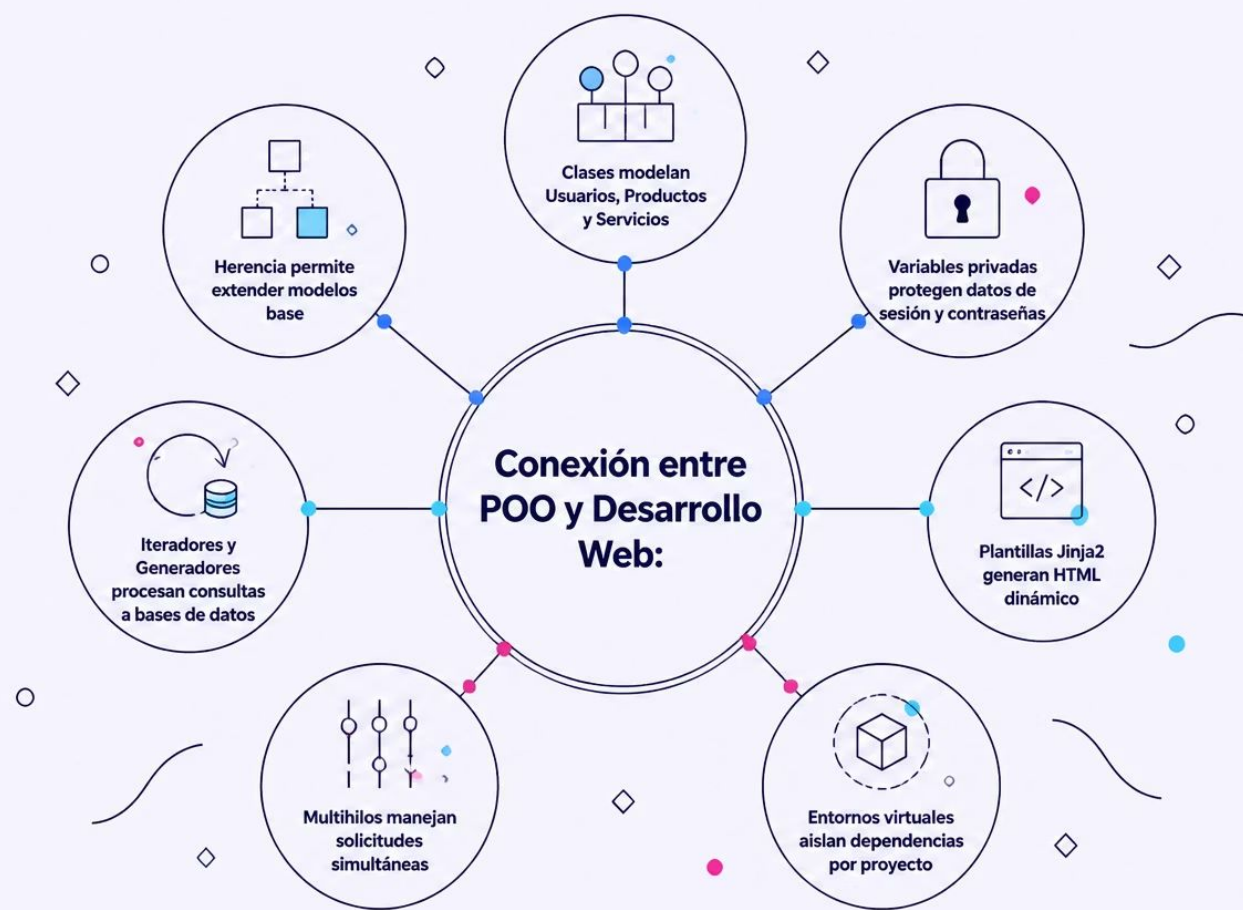
```
source  
mi_entorno/bin/activate
```

 Instalar

```
pip install flask
```

POO y el Desarrollo Web: La Conexión

Todos los conceptos de POO se aplican directamente al construir aplicaciones web modernas con frameworks como Flask o Django. El siguiente mapa muestra cómo cada elemento tiene su rol en un proyecto real.



Las clases son el núcleo del desarrollo web en Python. Todo lo demás –templates, hilos, entornos virtuales– orbita alrededor de ellas para construir aplicaciones robustas y profesionales.

Glosario de Conceptos Clave

Referencia rápida de los términos más importantes vistos en esta presentación.

Término	Definición
Clase	Plantilla utilizada para crear objetos con atributos y métodos.
Instancia	Objeto específico generado a partir de una clase.
Método	Función definida dentro de una clase que opera sobre sus datos.
Herencia	Mecanismo para que una clase obtenga características de otra.
Variable privada	Atributo protegido con doble guion bajo (__) para encapsular datos.
Iterador	Objeto que permite recorrer elementos secuencialmente con next().
Generador	Función que produce valores bajo demanda usando yield.
Plantilla (Template)	Documento que define la estructura visual de una página web dinámica.
Multihilo	Técnica para ejecutar múltiples tareas de forma simultánea.
Entorno virtual	Ambiente aislado donde se administran las dependencias de un proyecto Python.

📌 Dominar estos conceptos es el primer paso para desarrollar aplicaciones web profesionales con Python. ¡Practica con ejemplos reales en Flask o Django!